Universidade de Brasília

Departamento de Ciência da Computação

Relatório do Trabalho 1 de Programação Concorrente

Evelyn Soares Pereira 17/0102785

Brasília
10 de novembro de 2026

1 Introdução

Quando *threads* cooperam por memória compartilhada, acessos não coordenados a um mesmo dado geram *condições de corrida*: o resultado passa a depender da intercalação escolhida pelo escalonador e pode mudar de uma execução para outra. Este trabalho estuda um problema de Manna e Pnueli apresentado por Ben-Ari (2006, Apêndice C, exercício 8): calcular o coeficiente binomial $\binom{n}{k}$ com um processo para o numerador e outro para o denominador, que entrega resultados parciais para que as divisões sejam feitas imediatamente, mantendo pequenos os valores intermediários. A comunicação entre os dois é sincronizada com semáforos e implementada em C com POSIX Pthreads.

2 Formalização do Problema Proposto

2.1 Coeficiente Binomial e Resultados Parciais

Para inteiros $0 \le k \le n$,

$$\binom{n}{k} = \frac{n!}{(n-k)!\,k!} = \frac{n \cdot (n-1) \cdots (n-k+1)}{1 \cdot 2 \cdots k}.$$

Calcular o numerador inteiro e só dividir no final é inviável mesmo para valores modestos: em $\binom{30}{15}$, o numerador $30 \cdot 29 \cdots 16 \approx 2{,}0 \times 10^{20}$ não cabe em 64 bits, embora o resultado ($155\,117\,520$) caiba em 32. A proposta do exercício é intercalar as operações: no passo i, multiplicar o acumulado pelo i-ésimo fator do numerador e dividir pelo i-ésimo do denominador. Para $\binom{10}{3}$:

$$1 \xrightarrow{\times 10} 10 \xrightarrow{\div 1} 10 \xrightarrow{\times 9} 90 \xrightarrow{\div 2} 45 \xrightarrow{\times 8} 360 \xrightarrow{\div 3} 120. \tag{1}$$

Como a divisão inteira em C descarta o resto sem aviso, é preciso garantir que toda divisão intermediária é exata. Seja $P(i, j) = j \cdot (j+1) \cdots (j+i-1)$ o produto de i inteiros consecutivos a partir de j.

Lema 1. *Para todos os inteiros $i \ge 1$ e $j \ge 1$, $i!$ divide $P(i, j)$.*

Demonstração. Por indução em i. Para $i = 1$, $P(1, j) = j$ e $1! = 1$. Seja $i \ge 2$ e suponha que $(i-1)!$ divide $P(i-1, m)$ para todo $m \ge 1$. Colocando em evidência os fatores comuns $(j+1) \cdots (j+i-1) = P(i-1,\, j+1)$,

$$P(i, j+1) - P(i, j) = \big[(j+i) - j\big]\, P(i-1,\, j+1) = i \cdot P(i-1,\, j+1),$$

que, pela hipótese, é múltiplo de $i \cdot (i-1)! = i!$. Como $P(i, 1) = i!$ e cada passo de j para $j+1$ soma um múltiplo de $i!$, por indução em j todo $P(i, j)$ é múltiplo de $i!$. □

Após i multiplicações, o numerador acumulou $N_i = n(n-1) \cdots (n-i+1) = P(i,\, n-i+1)$, com $n-i+1 \ge 1$. Se o acumulado já foi dividido por $1, \ldots, i-1$, ele vale $N_i/(i-1)!$, e dividi-lo por i dá $N_i/i! = \binom{n}{i}$, que é inteiro pelo lema. Assim, o denominador só precisa enviar o fator i, e o maior valor intermediário é $i\binom{n}{i}$ (em $\binom{30}{15}$, cerca de $2{,}3 \times 10^9$).

2.2 Threads, Dados Compartilhados e Condições de Corrida

Além da *thread* Principal, que cria as outras e imprime o resultado após o `pthread_join`, há duas: o **Denominador**, que escreve os fatores $1, 2, \ldots, k$, em ordem, num *buffer* compartilhado de N posições, e o **Numerador**, que no passo i multiplica o seu acumulado (local) por $n - i + 1$, lê o próximo fator do *buffer* e divide por ele. O dado compartilhado é o *buffer*, junto com as variáveis que indicam quantos fatores estão disponíveis. Sem sincronização, há três condições de corrida: (1) **leitura antes da escrita**, em que o Numerador lê uma posição ainda não escrita e obtém um valor antigo (ou 0, dividindo por zero); (2) **sobrescrita**, em que o Denominador escreve sobre um fator ainda não lido, que se perde; e (3) **atualização perdida**, em que as duas alteram um mesmo contador (`qtd++` e `qtd--`), operação que não é atômica, e uma alteração apaga a outra. A Tabela 1 mostra uma execução de $\binom{5}{2} = 10$ com $N = 1$ em que (1) e (2) levam ao resultado 4, sem nenhum erro aparente.

Tabela 1: Execução sem sincronização de $\binom{5}{2}$, passo a passo.

Passo	Denominador	*buffer*	Numerador	Acumulado	Problema
1	–	–	multiplica por 5	5	
2	escreve 1	1	–	5	
3	escreve 2	2	–	5	o 1 se perdeu (corrida 2)
4	–	2	lê 2 e divide: 5/2	2	divisão truncada
5	–	2	multiplica por 4	8	
6	–	2	lê 2 de novo: 8/2	**4**	valor antigo (corrida 1)

Uma solução correta deve garantir que cada fator seja lido exatamente uma vez e na ordem em que foi escrito, exclusão mútua no acesso ao *buffer* e término sem *deadlock* nem *starvation*, preferencialmente sem espera ocupada.

3 Descrição do Algoritmo Desenvolvido para Solução do Problema Proposto

3.1 Mecanismo de Sincronização

A solução usa dois semáforos POSIX e um *buffer* circular de N posições: `posicoes_livres`, que começa em N e conta as posições que podem ser escritas, e `fatores_prontos`, que começa em 0 e conta os fatores escritos e ainda não lidos. Um semáforo é um contador de operações atômicas que bloqueia a *thread* quando chega a zero, sem espera ocupada, e isso corresponde exatamente às duas esperas do problema: o Denominador espera uma posição livre (evitando a corrida 2) e o Numerador espera um fator pronto (evitando a corrida 1); a corrida 3 desaparece porque o contador passa a ser o próprio semáforo. Cada semáforo recebe `sem_wait` de uma *thread* e `sem_post` da outra, ou seja, é usado para sinalização. Não é preciso *mutex*: cada índice do *buffer* é usado por uma única *thread*, n, k e N são constantes, e o resultado só é lido depois do `pthread_join`.

3.2 Pseudocódigo

```
Denominador:
  escrita = 0
  para i de 1 até k:
    sem_wait(posicoes_livres)
    buffer[escrita] = i
    escrita = (escrita + 1) mod N
    sem_post(fatores_prontos)

Principal:
  sem_init(posicoes_livres, N)
  sem_init(fatores_prontos, 0)
  cria Denominador e Numerador
  espera as duas (pthread_join)
  imprime resultado
```

```
Numerador:
  leitura = 0
  acumulado = 1
  para i de 1 até k:
    acumulado = acumulado * (n - i + 1)
    sem_wait(fatores_prontos)
    fator = buffer[leitura]
    leitura = (leitura + 1) mod N
    sem_post(posicoes_livres)
    acumulado = acumulado / fator
  resultado = acumulado
```

No código, cada `sem_wait` é precedido de um `sem_trywait` que só serve para imprimir um aviso quando a *thread* vai bloquear (o efeito é o mesmo), e há `printf`s e `sleep`s que mostram a execução e simulam o tempo de cada operação.

3.3 Corretude

Sejam L e P os valores de `posicoes_livres` e `fatores_prontos`, e $a_d, a_n \in \{0, 1\}$ iguais a 1 enquanto, respectivamente, o Denominador ou o Numerador já passou do `sem_wait` mas não do `sem_post` seguinte. Cada operação apenas transfere uma unidade ($L \to a_d \to P \to a_n \to L$), então vale sempre o invariante $L + P + a_d + a_n = N$.

Sem condições de corrida. O Numerador só lê o i-ésimo fator depois de i `sem_post`s em `fatores_prontos`, ou seja, depois de ele ser escrito. O Denominador só escreve o j-ésimo fator, na posição $(j-1) \bmod N$, depois de pelo menos $j - N$ `sem_post`s em `posicoes_livres`, ou seja, depois que o fator $j - N$, da mesma posição, foi lido. Como `sem_post` e `sem_wait` também sincronizam a memória no POSIX, cada fator é lido uma vez e em ordem, o Numerador refaz a sequência (1) e, pelo lema, obtém $\binom{n}{k}$.

Sem deadlock. As duas *threads* só bloqueariam juntas com $L = a_d = 0$ (*buffer* cheio) e $P = a_n = 0$ (*buffer* vazio), o que daria $0 = N \geq 1$: o *buffer* não pode estar cheio e vazio ao mesmo tempo.

Sem starvation e com término. Se o Denominador está bloqueado, o invariante dá $P + a_n = N \geq 1$: o Numerador tem um fator na mão ou um pronto para ler e, em um número finito de passos, faz o `sem_post` que o libera; o caso do Numerador bloqueado é simétrico. Ninguém espera uma *thread* que já terminou: se o Numerador terminou, o Denominador, na sua j-ésima espera, encontra $L = N - (j-1) + k > 0$, e o caso simétrico dá $P \geq 1$. Como cada semáforo tem um único possível esperador, o `sem_post` acorda justamente a *thread* bloqueada, e, com laços de k iterações, as duas terminam (supondo um escalonador justo).

3.4 Implementação e Testes

O código está em `coef_binomial.c` e é compilado com `gcc -Wall -o coef_binomial coef_binomial.c -lpthread`. Os valores de n, k e N são constantes no início do código (padrão $n = 10$, $k = 5$ e $N = 2$, com $N < k$ para que a espera do Denominador apareça); para outro caso, basta alterá-las e compilar de novo. O acumulado é `unsigned long long`, e o programa avisa se um valor intermediário passar de 64 bits. A Tabela 2 traz

casos que podem ser testados; todos foram executados e deram o resultado esperado. Por causa dos `sleep`, cada execução leva cerca de $4k$ segundos.

Tabela 2: Casos de teste sugeridos.

n	k	N	$\binom{n}{k}$ esperado	O que o caso mostra
10	5	2	252	padrão: o *buffer* enche e o Denominador espera
10	3	5	120	exemplo do enunciado, Equação (1)
5	2	1	10	caso da Tabela 1, agora correto
7	0	2	1	$k = 0$: nenhuma *thread* entra no laço
7	7	2	1	$k = n$
30	15	5	155 117 520	o numerador completo estouraria 64 bits
100	5	5	75 287 520	n grande e k pequeno

A Figura 1 mostra a execução padrão: o Denominador enche o *buffer* e espera até o Numerador liberar uma posição, e o acumulado após cada divisão é $\binom{10}{i}$: 10, 45, 120, 210 e 252.

```
Calculando C(10, 5) com buffer de 2 posições

Denominador: escrevi o fator 1 na posição 0
Numerador: multipliquei por 10, acumulado = 10
Numerador: li o fator 1 da posição 0
Denominador: escrevi o fator 2 na posição 1
Denominador: escrevi o fator 3 na posição 0
Denominador: buffer cheio, vou esperar
Numerador: dividi por 1, acumulado = 10
Numerador: multipliquei por 9, acumulado = 90
Numerador: li o fator 2 da posição 1
Denominador: escrevi o fator 4 na posição 1
Denominador: buffer cheio, vou esperar
Numerador: dividi por 2, acumulado = 45
Numerador: multipliquei por 8, acumulado = 360
Numerador: li o fator 3 da posição 0
Denominador: escrevi o fator 5 na posição 0
Denominador: terminei
Numerador: dividi por 3, acumulado = 120
Numerador: multipliquei por 7, acumulado = 840
Numerador: li o fator 4 da posição 1
Numerador: dividi por 4, acumulado = 210
Numerador: multipliquei por 6, acumulado = 1260
Numerador: li o fator 5 da posição 0
Numerador: dividi por 5, acumulado = 252
Numerador: terminei

Resultado: C(10, 5) = 252
```

Figura 1: Saída da execução padrão ($n = 10$, $k = 5$, $N = 2$).

4 Conclusão

O trabalho modelou o cálculo de $\binom{n}{k}$ proposto por Ben-Ari como uma comunicação produtor/consumidor entre as *threads* Denominador e Numerador, identificou três condições de corrida no *buffer* compartilhado e as eliminou com apenas dois semáforos, com argumentos de corretude, ausência de *deadlock* e de *starvation*. O lema de divisibilidade garante que as divisões intermediárias são exatas e mantêm os valores pequenos. A principal limitação é o tamanho do inteiro: com 64 bits, casos como $\binom{67}{33}$, cujo resultado cabe mas um valor intermediário não, são recusados; uma biblioteca de precisão arbitrária, como a GMP, removeria esse limite. Já usar mais *threads* não ajudaria: cada passo do Numerador depende do anterior, e os fatores precisam chegar em ordem.

5 Referências

BEN-ARI, Mordechai. **Principles of Concurrent and Distributed Programming**. 2. ed. Harlow: Addison-Wesley, 2006. ISBN 978-0-321-31283-9.